A program called a **compiler** is used to transform the high-level source code directly into machine code. This requires the compiler to *assign* hardware memory locations to each of the abstract variables being referenced. For example, the first time the compiler encounters the variable *A* in Table 4-5 (line 100), it understands that the programmer is using this symbol to mean a single precision floating point variable. Correspondingly, the compiler designates four bytes of memory that will be used for nothing but to hold the value of this variable. Each subsequent time that an *A* appears in the program, the computer knows to update the value of the four bytes as needed. The compiler also breaks complicated mathematical expressions, such as: $Y = LOG(X^{COS(Z)})$, into more basic arithmetic. Microprocessors only know how to add, subtract, multiply and divide. Anything more complicated must be done as a series of these four elementary operations.

High-level languages isolate the programmer from the hardware. This makes the programming much easier and allows the source code to be transported between different types of microprocessors. Most important, the programmer who uses a compiled language needs to know *nothing* about the internal workings of the computer. Another programmer has assumed this responsibility, the one who wrote the compiler.

Most compilers operate by converting the *entire* program into machine code *before* it is executed. An exception to this is a type of compiler called an **interpreter**, of which **interpreter BASIC** is the most common example. An interpreter converts a *single line* of source code into machine code, executes that machine code, and then goes on to the next line of source code. This provides an interactive environment for simple programs, although the execution speed is extremely slow (think a factor of 100).

The highest level of programming sophistication is found in **applications** packages for DSP. These come in a variety of forms, and are often provided to support specific hardware. Suppose you buy a newly developed DSP microprocessor to embed in your current project. These devices often have lots of built-in features for DSP: analog inputs, analog outputs, digital I/O, antialias and reconstruction filters, etc. The question is: how do you program it? In the worst case, the manufacturer will give you an *assembler*, and expect you to learn the internal architecture of the device. In a more typical scenario, a C *compiler* will be provided, allowing you to program without being bothered by how the microprocessor actually operates.

In the best case, the manufacturer will provide a sophisticated software package to help in the programming: libraries of algorithms, prewritten routines for I/O, debugging tools, etc. You might simply connect icons to form the desired system in an easy-to-use graphical display. The *things* you manipulate are signal pathways, algorithms for processing signals, analog I/O parameters, etc. When you are satisfied with the design, it is transformed into suitable machine code for execution in the hardware. Other types of applications packages are used with image processing, spectral analysis, instrumentation and control, digital filter design, etc. This is the shape of the future.